Latency-Constrained Dark Pool Liquidity Routing: A Time-Expanded Graph Approach

Tianchi He

March 14, 2026

1 Problem Description

In modern high-frequency and quantitative trading, institutional investors frequently need to execute massive block trades (the Source, s) to public exchanges (the Sink, t) without heavily impacting public market prices. To achieve this, liquidity is routed through a fragmented, directed inter-dealer network of "dark pools," mathematically modeled as a graph $G = (V, E)$.

Each edge $e \in E$ possesses a bandwidth capacity c_e and a discrete transmission latency τ_e, measured in microseconds. While standard network flow paradigms—such as the Edmonds-Karp or Dinic's algorithms—expertly handle static spatial capacities, the financial reality introduces two profound temporal and physical constraints:

1. **Strict Latency Bound (K):** Financial markets are highly volatile. The entire execution must completely clear the network within a strict maximum time window of K microseconds. Flow arriving at time $t > K$ is inherently invalid.
2. **Vertex Processing Capacities (C_v):** Regulatory compliance and physical matching-engine throughput place strict upper bounds on the volume of shares a specific node v can process per microsecond, regardless of its incoming network bandwidth.

The objective is to deduce the absolute maximum flow from s to t such that no routed share violates the latency bound K, and no intermediate node v processes more than C_v shares at any single discrete time step.

2 Real-Life Application Background

This formulation rigorously mirrors the microstructure of Alternative Trading Systems (ATS) and algorithmic execution strategies (e.g., VWAP or TWAP). When a routing engine partitions large orders, it must continuously balance shear volume against fragmented liquidity.

If an algorithm is blind to a node's internal processing capacity, it will cause a "temporal traffic jam," where orders are queued and heavily penalized by latency. Furthermore, real-world broker-dealers permit shares to temporarily "rest" in limit order books while waiting for outbound network bandwidth to become available. Standard routing algorithms fail to capture this temporal storage. The architecture proposed herein natively encodes both matching engine bottlenecks and resting liquidity directly into its geometric structure.

3 Algorithm Design: The TEG Reduction

To circumvent the limitations of static flow networks, we apply a profound topological reduction. We transform the dynamic, rules-heavy 2D graph G into a static 4-dimensional Time-Expanded Graph (TEG), denoted as $G' = (V', E')$. To track a share in G', we pinpoint its exact location across Space, Time, Internal State, and Flow Magnitude.

3.1 Temporal Replication & Vertex Splitting

To enforce the vertex capacities C_v, we cannot simply cap edges. Instead, we fracture the network across time and apply vertex splitting. For every physical vertex $v \in V$ and every discrete microsecond $k \in \{0, \dots, K\}$, we instantiate two temporal nodes: an *in-node* $v_{k,in}$ and an *out-node* $v_{k,out}$.

We draw a directed internal edge $(v_{k,in} \to v_{k,out})$ with a strict capacity of C_v. Because any flow passing through node v at time k must physically traverse this internal edge, the matching engine bottleneck is flawlessly enforced by standard max-flow mathematics.

3.2 Resting Liquidity & Spatial Connections

To model the order book's temporal storage, we create "wait edges" connecting a node's output to its own future input: $(v_{k,out} \to v_{k+1,in})$ with infinite capacity ∞. This permits shares to rest temporally without moving spatially.

Simultaneously, spatial network connections (u, v) with latency τ and capacity c are projected diagonally across time as $(u_{k,out} \to v_{k+\tau,in})$ with capacity c.

3.3 Super-Node Initialization

To satisfy the single-source/single-sink requirement of Max-Flow algorithms, we introduce a Super-Source S^* connected exclusively to the institution's starting ledger, $s_{0,in}$, with capacity ∞. We then introduce a Super-Sink T^*, drawing edges from every valid temporal manifestation of the market, $t_{k,out}$ (where $k \leq K$), with capacity ∞. This successfully captures all routed liquidity regardless of its exact arrival time.

4 Structural Rationale & Data Structures

The elegance of the TEG is that it absorbs all dynamic market rules into pure topology. Once compiled, an Edmonds-Karp solver can blindly traverse G' without evaluating any custom logic for deadlines or regulatory caps.

However, G' possesses a massive hypothetical vertex space $|V'| = O(NK)$. A standard $V' \times V'$ adjacency matrix would allocate $O(N^2K^2)$ memory. Because financial networks are sparse and time flows strictly unidirectionally, over 99% of this matrix would be zeroes, causing catastrophic cache misses during Breadth-First Search (BFS).

To achieve optimal memory efficiency, we implement a **Dictionary-Backed Adjacency List**: an array of hash maps (`list[dict]`). This brilliantly preserves the $O(1)$ edge-weight lookups and updates required for tracking residual capacities, while strictly allocating memory bounded to $O(MK)$ exclusively for edges that physically exist.

5 Pseudocode

Algorithm 1 Latency-Constrained TEG Routing

Require: Graph $G(V, E)$, Latency Bound K, Node Caps C_v

Initialize `graph` as Array of Hash Maps
$S^* \leftarrow$ SuperSource, $T^* \leftarrow$ SuperSink
for $v \in V, k \in [0, K]$ **do**
    AddEdge$(v_{k,in}, v_{k,out}, C_v)$ ▷ Matching Engine
end for
for $v \in V, k \in [0, K-1]$ **do**
    AddEdge$(v_{k,out}, v_{k+1,in}, \infty)$ ▷ Resting Orders
end for
for $(u, v, c, \tau) \in E$ **do**
    for $k \in [0, K-\tau]$ **do**
        AddEdge$(u_{k,out}, v_{k+\tau,in}, c)$ ▷ Network Transit
    end for
end for
AddEdge$(S^*, s_{0,in}, \infty)$ ▷ Institutional Injection
for $k \in [0, K]$ **do**
    AddEdge$(t_{k,out}, T^*, \infty)$ ▷ Market Capture
end for
return EdmondsKarp(`graph`, S^*, T^*)

6 Rigorous Complexity Analysis

Let $N = |V|$ and $M = |E|$ of the original static graph G.

6.1 Space Complexity

The TEG introduces $2(K + 1)$ nodes for each of the N physical vertices, yielding exactly $|V'| = O(NK)$. The edges comprise internal vertex-splits $O(NK)$, temporal wait edges $O(NK)$, and spatial projections $O(MK)$. The total edge count is bounded by $|E'| = O(MK + NK) = O(MK)$. By utilizing the dictionary-backed adjacency list, the space complexity avoids the quadratic penalty and remains strictly tightly bounded to $\mathcal{O}(MK)$.

6.2 Time Complexity

The graph construction phase requires iterating over nodes and edges across K time steps, strictly bounding compilation time to $\mathcal{O}(MK)$.

The Edmonds-Karp execution utilizes Breadth-First Search (BFS) to find the shortest augmenting paths in terms of edge count. The fundamental time complexity of Edmonds-Karp on any network is $\mathcal{O}(|V'| \cdot |E'|^2)$. Substituting our specific topological bounds:

$$\mathcal{O}((NK) \cdot (MK)^2) = \mathcal{O}(NM^2K^3)$$

While the cubic dependence on the temporal bound K appears computationally expensive, the practical runtime is drastically lower. In high-frequency trading environments, K is strictly constrained (often representing milliseconds or microseconds). Furthermore, because the initial topology of G' is a pure Directed Acyclic Graph (DAG) where flow only moves forward in time, the BFS wavefront expands with extreme efficiency, resulting in significantly fewer necessary augmenting paths than a heavily cyclical graph would demand.